

Decision Document

Part [A]: decisions made along the way. Part [B]: one feature I would add next.

Lara Longhi

[A] Decisions made along the way

Product and interaction decisions

I treated the five screens as one continuous decision flow: **lander** → **budget** → **dietary needs** → **nutritional goals** → **weekly plan**. All onboarding screens share the same layout primitives (*OnboardingScreen*, progress header, option grid) so the flow feels consistent rather than like five unrelated pages.

- The budget slider covers the required €25–€150 range and defaults to €82 as a practical mid-point.
- Dietary needs and nutritional goals are **multi-select** (restrictions combine , e.g. vegetarian + dairy-free). Continue is disabled until a choice is explicit, so a plan is never silently generated from incomplete input.
- The weekly-plan screen uses a real recipe card instead of the placeholder bars from the reference: meal name, prep time, servings, cost, ingredients and step-by-step instructions are all visible without leaving the screen. A shopping-list view was added because it turns the generated plan into something immediately actionable, with prices pulled from the supplied catalog.

Weekly-plan interactions I added beyond the reference

The brief leaves everything inside the daily card up to me. Beyond the base recipe/shopping-list content, I added four interactions that make the plan feel usable across more than a single session, rather than a static, throwaway result:

- **A real, consolidated shopping list.** Rather than 14 separate per-recipe ingredient lists, every ingredient across the whole week is aggregated by catalog product: compatible quantities are summed (e.g. 200 g + 0.5 kg → 700 g) and priced against the whole packs actually needed. The user gets one flat, priced list to shop from at the supermarket, instead of cross-referencing every meal's ingredients by hand.
- **Favorite a recipe, for future plans.** Tapping the heart on a meal card marks it as a favorite (stored by name in the app state). Favorited dishes aren't just remembered , the scheduler gives them a small preference bump on the next plan generation, so a recipe the user liked is more likely to reappear in a later week instead of the app only ever proposing brand-new dishes.
- **Remove a disliked ingredient, plan regenerates around it.** From the shopping list, any ingredient can be removed. The app adds that product to the exclusion list and calls generation again with the same budget and preferences; only the recipe(s) that genuinely required that ingredient drop out, everything else in the week is kept as close to unchanged as the new search allows.
- **Visit the original recipe.** Every meal card links out to the exact publisher page the recipe was sourced from (“View full recipe ↗”), so the user can always read the full method, see the original photos, and trust the recipe wasn't invented by the model. If the link fails to open, the card shows an inline retry message instead of failing silently.

Pricing logic: proportional per meal vs. whole-pack for the shopping list

The two prices shown in the app are computed differently on purpose, and are not meant to reconcile line by line:

- **Per meal: proportional cost.** Each recipe card shows the value of what that recipe actually consumes: for every ingredient, the app takes the fraction of a catalog product's pack the recipe needs (e.g. 125 g out of a 500 g pasta pack = 25%) and multiplies it by that pack's list price (*lib/cost.ts*). Summed across a recipe's ingredients, this gives a fair, real-valued cost-per-recipe useful for comparing dishes, and it's the number the budget-aware scheduler relies on while

composing a week, since it needs a smooth value to fill the budget slot by slot rather than jumping in whole-pack increments.

- **Shopping list: whole-pack cost.** You can't buy a quarter of a pasta box at checkout, so the shopping list works differently: it first sums every recipe's fractional usage of a given product across the entire week, then rounds that total **up** to the nearest whole number of packs (minimum 1, plus one extra pack for any ingredient the source left unspecified, e.g. “salt to taste”), and prices it at the full per-pack price (*lib/shopping.ts*). This is the number meant to match what the user would actually pay at checkout.
- Because packs are indivisible, the shopping-list total is always $\geq$ the sum of the 14 individual meal prices, that gap is intentional, not a bug. It's also why the scheduler checks the budget against the rounded, whole-pack shopping-list total while it composes a week, rather than against the sum of proportional meal prices, so the final plan never quietly exceeds the user's stated budget once packs get rounded up.

Design decisions and deviations from Figma

The Figma layouts were reproduced with shared React Native components, the supplied assets, the Promo type system, and the reference colors/spacing. The iPhone status bar and Dynamic Island are left to the OS rather than drawn as app content. On desktop/web, the app uses a centered canvas capped at 480pt, since the source design only specifies mobile frames. The most visible intentional deviations from the five-screen reference:

Reference screen	Delivered implementation	Reason
Lander	Kept title, bag composition, green CTA and entrance animation, but used the available local food PNGs instead of the exact Figma emoji layers.	Assets in the middle were explicitly left free to swap; the local PNGs were more reliable across platforms and kept the animation functional.
Budget selection	Added the missing €25–€150 slider, with a large budget readout and a default of €82.	The slider input was explicitly missing from the design; a native control was chosen for touch usability, with a keyboard-accessible HTML range on web.
Dietary needs	Kept the two-column option grid; added real multi-select behavior, selected styling and disabled/enabled Continue.	The design shows the options but not the interaction model; dietary restrictions can naturally be combined (e.g. vegetarian + dairy-free).
Nutritional goals	Kept the two-column grid; wired selection to generation, with loading and visible error states.	A static screen would give no feedback if the generation service is unavailable.
Weekly meal plan	Replaced the gray placeholder lines with real recipe cards (metadata, ingredients, quantities, steps); kept day navigation but used chronological Mon–Sun order; added a cost summary and a shopping-list view.	The brief requires an actionable 7-day plan and a shopping list tied to the priced catalog. The reference's swapped Wed/Thu order read as a visual inconsistency rather than intended product behavior.

Beyond the table: option tiles, day tabs, buttons and sliders all carry explicit **selection / disabled semantics** rather than relying on color alone, and functional loading, error and empty states were added throughout, the design mainly showed the happy path, but recipe generation depends on network and model availability in a real deployment.

Data and LLM workflow decisions

- **Server-side generation, not fully client-side.** The brief suggests everything can be done client-side for simplicity; I moved meal-plan generation behind a small API route (*app/api/meal-plan+api.ts*) instead, so the OpenAI key never ships inside the client bundle. Given the take-home explicitly hands over a private API key, I treated not leaking it as a higher priority than strict client-only simplicity, and kept the route thin (validation, rate-limiting to 2 concurrent requests, no state beyond an in-memory cache).
- **The catalog is the source of truth, not the model.** The LLM cannot invent an unconstrained recipe list: the workflow filters eligible catalog products first, discovers candidate recipes on the web, extracts structured data from the publisher's own *schema.org/Recipe* markup, and only then asks the model to match every required ingredient against real catalog candidates. Recipes that omit a required ingredient, substitute an incompatible product, violate a selected preference, or push the plan over budget are rejected outright rather than adjusted.
- **Budget- and variety-aware scheduling.** The scheduler targets 14 meals (lunch + dinner $\times$ 7 days), requires at least 7 distinct dishes and allows at most 2 repeats of any recipe, then falls back to one meal per day if two meals per day cannot fit the budget. If even that is not achievable, the app reports the limitation instead of fabricating a plan or silently dropping ingredients.
- Verified recipe results are cached in server memory for 6 hours to reduce repeated search cost while still allowing fresh discovery on later requests.

[B] One feature I would add next

Replace-one-meal regeneration

A user should be able to tap “*Replace*” on a single meal card, optionally add a short reason, “*less expensive*”, “*no fish*”, “*use what I already have*”, and regenerate only that meal while the rest of the week stays untouched. The replacement would run through the exact same pipeline already in place (catalog filtering → web discovery → source matching → validation), but with the cost of the other 13 meals reserved up front so the new pick still fits the weekly budget.

Why this one, specifically:

- Meal plans are personal, and a single disliked or inconvenient meal shouldn't force the user back through the whole onboarding flow, as the current app requires.
- It is the highest-leverage way to show the LLM workflow is controllable and safe under a **partial** update, not just a good result on a first, clean generation.
- It composes naturally with a feature that already exists: excluding a product from the shopping list already regenerates the affected recipe. Extending that same mechanism to an explicit, user-initiated per-meal replace is a small, coherent step rather than a new subsystem.